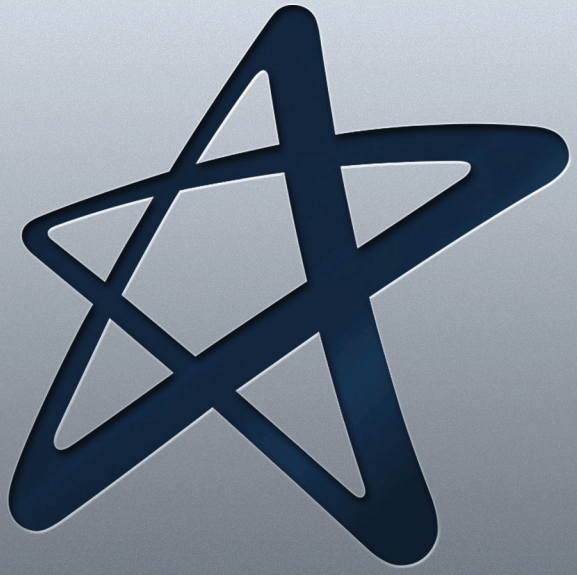
Programação para Dispositivos Móveis



Cruzeiro do Sul Virtual
Educação a Distância

Material Teórico

Consumindo JSON em aplicações Android

Responsável pelo Conteúdo:
Prof. Ms. Luis Carlos Reis

Revisão Textual:
Prof. Ms. Douglas Almendro

Revisão Textual:
Profa. Ms. Luciene Oliveira da Costa Santos

UNIDADE Consumindo JSON em aplicações Android

- Introdução
- Consumindo o Json
- Objetos Http
- Conclusão



OBJETIVO DE APRENDIZADO

- Demonstrar alguns recursos sobre JSON interessantes disponíveis em aplicações Android.

ORIENTAÇÕES

Caro(a) aluno(a),

Estamos começando nossos estudos JSON e outros recursos visuais.

Para obter um bom aproveitamento, nesta unidade, vamos conferir sua estrutura:

Conteúdo Teórico: neste link, você encontrará o material principal de estudos na forma de texto escrito.

Atividade de Sistematização: os exercícios disponibilizados são de autocorreção para que você pratique o que aprendeu na disciplina e identifique os pontos em que precisa prestar mais atenção, ou pedir esclarecimentos a seu tutor. Além disso, as notas atribuídas aos exercícios serão parte de sua média final na disciplina.

Atividade de Aprofundamento: a atividade disponibilizada está apresentada para aplicação ou debate.

Material Complementar e Referências Bibliográficas: nestes links, você poderá ampliar seus conhecimentos.

Lembramos a você da importância de realizar todas as atividades propostas dentro do prazo estabelecido para cada unidade. Isso evitará que o conteúdo se acumule e que você tenha problemas ao final do semestre.

Uma última recomendação: caso tenha problemas para acessar algum item da disciplina, ou dúvidas em relação ao conteúdo, não deixe de entrar em contato com seu professor-tutor por meio do botão mensagens.

Bom estudo!

Contextualização

Ao desenvolver uma aplicação, temos que levar em conta que a aparência visual do aplicativo é muito importante e também a sua interface com o mundo web. Temos que ter em mente que, com a ajuda desses recursos, podemos melhor e facilitar a apresentação e interação do usuário com o aplicativo.

Nesta unidade, será explicada e demonstrada a utilização dos recursos do JSON e recursos web.

Bom estudo!

Introdução

Nesta aula, iremos utilizar o JSON em aplicações Android. O JSON (JavaScript Object Notation) nada mais é do que uma estrutura para representação dados em JavaScript.

A marca de objetos JavaScript (JSON) possui uma formatação bastante prática para troca de dados. Ela é simples de ler e de escrever, e também é simples de implementar. Essa marcação de objetos está baseada em um subconjunto de linguagens de programação que são próximas às linguagens C, C++, C#, Java, JavaScript, Python e muitas outras.

JSON é uma estrutura muito simples de trabalhar, exatamente o que precisamos para o desenvolvimento mobile. Com ele podemos representar uma **entidade/objeto** em um arquivo de formato fácil, que poderá ser lido e absorvido por diversas plataformas. Apesar de XML também ser um formato com características semelhantes, o JSON ganhou popularidade na Web pela sua associação com javascript, e é amplamente utilizado na comunicação de uma aplicação mobile com os websites.

O JSON é nativo do Android, que já vem com as classes necessárias para a sua implementação. São elas:

- **JSONObject** – que é uma coleção de pares do tipo chave/valor.
- **JSONArray** – que é uma sequência de valores.
- **JSONStringer** – que é um objeto usado para a criação de arquivos com dados no formato JSON.
- **JSONException** – que é o tratamento da exceção quando alguma estrutura do JSON falha.

O JSON além de ser um formato leve para troca de dados é também muito simples de ler. Mas quando dizemos que algo é simples, é interessante compará-lo com algo mais complexo para entendermos tal simplicidade. Nesse caso, podemos comparar o JSON com o formato XML.

Vamos visualizar esta diferença

XML

```
<?xml version="1.0" encoding="UTF-8"?>
  <id>1id>
  <nome>Alexandre Gamanome>
  <endereco>R. Qualquerendereco>
```

JSON

```
{"id":1,"nome":"Alexandre Gama", "endereco":"R. Qualquer"}
```

Figura 1

Visualmente, o segundo trecho (em JSON) é mais fácil de ler. Mas só existe essa diferença entre os dois? Na verdade, não. Podemos listar algumas outras vantagens:

Vantagens do JSON:

- Leitura mais simples;
- Analisador (parsing) mais fácil;
- JSON suporta objetos! Sim, ele é tipado;
- Velocidade maior na execução e transporte de dados;
- Arquivo com tamanho reduzido;
- Quem utiliza? Google, Facebook, Yahoo!, Twitter...

Iremos acessar o JSON da BleckBerry da seguinte página: https://goo.gl/bDXJLI. Perceba que esse JSON tem os seguintes dados:

```
[
    {
        “vehicleType”: “excavator”,
        “vehicleColor”: “yellow”,
        “fuel”: “diesel”,
        “approvedOperators”:
        [
            {
                “name”: “Greg Stark”,
                “experiencePoints”: 13
            },
            {
                “name”: “Aarti Patel”,
                “experiencePoints”: 21
```

```
            },
            {
                “name”: “Tarek Mizan”,
                “experiencePoints”: 9
            }
        ],
        “treadType”: “plate chain”
    },
    {
        “vehicleType”: “Dump Truck”,
        “vehicleColor”: “yellow”,
        “fuel”: “gasoline”,
        “approvedOperators”:
        [
            {
                “name”: “Marc Gervais”,
                “experiencePoints”: 8
            },
            {
                “name”: “Atsuko Iwamoto”,
                “experiencePoints”: 3
            },
            {
                “name”: “Chadwick Price”,
                “experiencePoints”: 54
            }
        ],
        “treadType”: “wheel”
    },
    {
        “vehicleType”: “Grader”,
        “vehicleColor”: “green”,
        “fuel”: “diesel”,
        “approvedOperators”:
        [
            {
                “name”: “Arun Verma”,
                “experiencePoints”: 34
            },
            {
                “name”: “Morgan Parker”,
                “experiencePoints”: 56
            },
            {
                “name”: “Paul Turner”,
                “experiencePoints”: 17
            }
        ],
        “treadType”: “wheel”
    }
]
```

Para exemplificar a utilização do JSON, iremos criar um novo projeto Android, no qual daremos o nome de *ExemploJson*. Neste projeto, iremos consumir o JSON da BlackBerry descrito acima, no qual o mesmo representa tipos de veículos. Perceba que o mesmo é formado por tipo de veículo, cor e tipo de combustível.

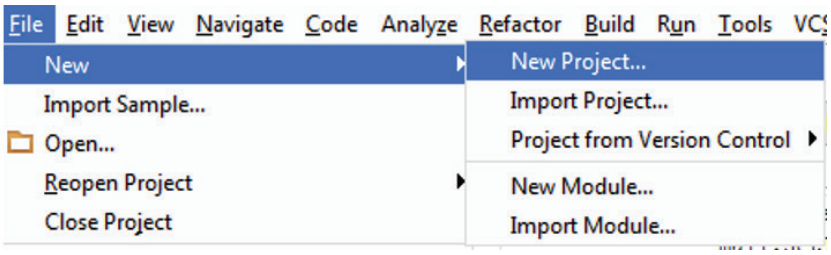


Figura 2

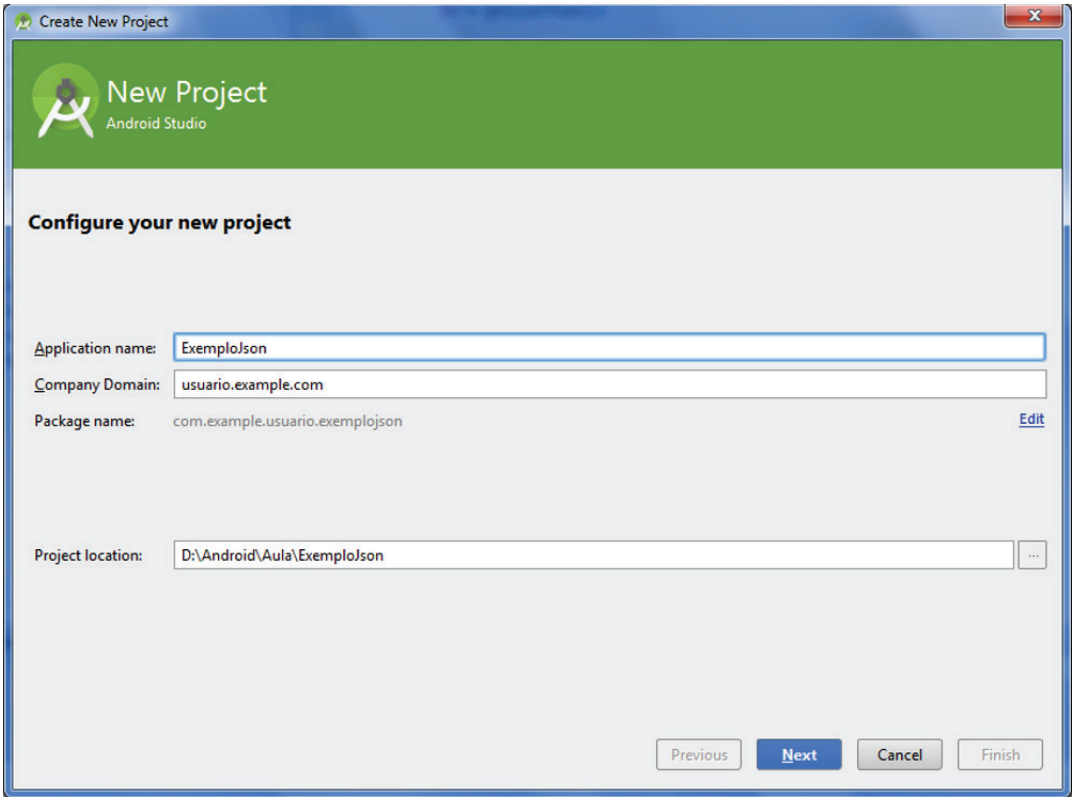


Figura 3

O nome da Activity pode será a padrão: MainActivity.

Consumindo o Json

Teremos que declarar no nosso **AndroidManifest.xml** a permissão de acesso à internet, caso contrário, a aplicação irá encerrar a cada vez que for solicitada uma conexão com a web. Para isso, abra o **AndroidManifest.xml** e deixe-o igual ao XML abaixo. Note que foi acrescentada a permissão para acesso à Internet.

```
<?xml version=”1.0” encoding=”utf-8”?>
<manifest xmlns:android=”http://schemas.android.com/apk/res/
android”
    package=”com.example.usuario.exemplojson” >

    <uses-permission android:name=”android.permission.INTER-
NET”/>

    <application
        android:allowBackup=”true”
        android:icon=”@mipmap/ic_launcher”
        android:label=”@string/app_name”
        android:theme=”@style/AppTheme” >
        <activity
            android:name=”.MainActivity”
            android:label=”@string/app_name” >
            <intent-filter>
               <action android:name=”android.intent.action.
MAIN” />

               <category android:name=”android.intent.catego-
ry.LAUNCHER” />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Iremos modificar o layout para que seja exibido o JSON retornado em uma lista. Portanto, iremos modificar nosso layout principal *activity_main.xml* com o layout abaixo:

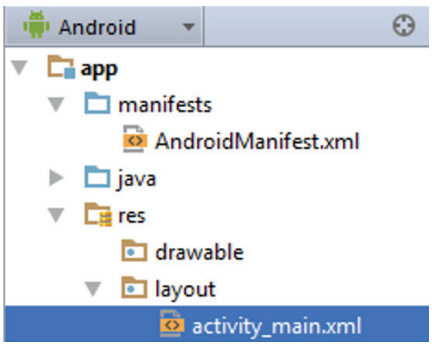


Figura 4

```
<RelativeLayout xmlns:android=”http://schemas.android.com/
apk/res/android”
    xmlns:tools=”http://schemas.android.com/tools”
android:layout_width=”match_parent”
    android:layout_height=”match_parent”
android:paddingLeft=”@dimen/activity_horizontal_margin”
   android:paddingRight=”@dimen/activity_horizontal_margin”
   android:paddingTop=”@dimen/activity_vertical_margin”
    android:paddingBottom=”@dimen/activity_vertical_margin”
tools:context=”.MainActivity”>

    <ListView
        android:id=”@android:id/list”
        android:layout_width=”fill_parent”
        android:layout_height=”wrap_content”/>

</RelativeLayout>
```

O mesmo ficará com o seguinte layout:



Figura 5

Iremos criar novo layout que irá receber os dados do JSON. Perceba que, neste layout, estamos configurando os campos: tipo de veículo, cor e tipo de combustível.

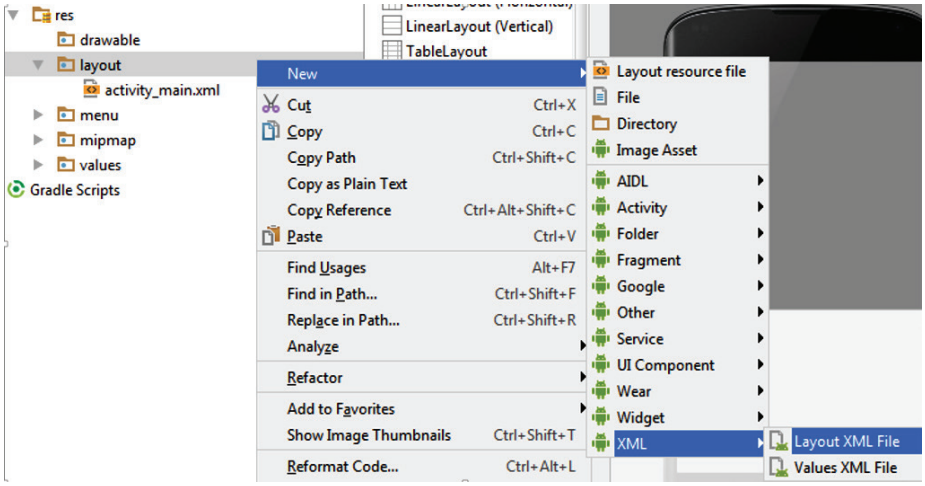


Figura 6

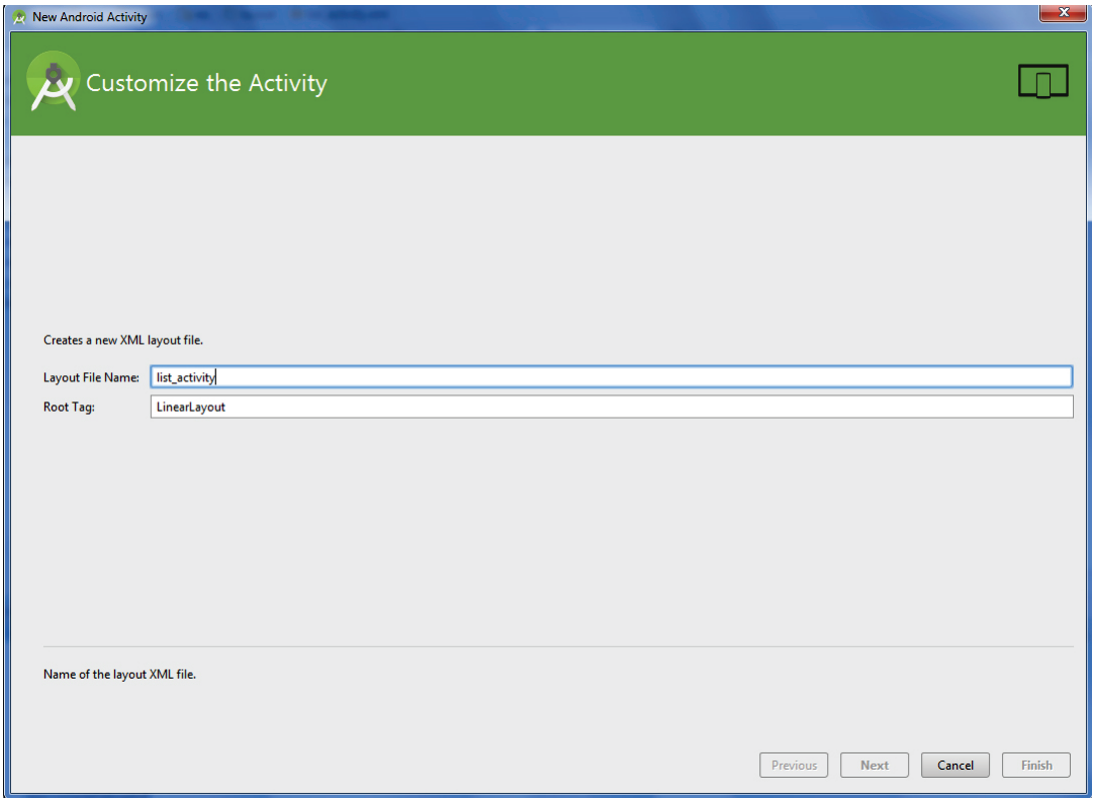


Figura 7

Daremos o nome de list_activity.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/
res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <LinearLayout
        xmlns:android="http://schemas.android.com/apk/res/an-
droid"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <TextView
            android:id="@+id/vehicleType"
            android:layout_width="fill_parent"
           android:layout_height="wrap_content"
            android:textSize="16sp"
            android:textStyle="bold"
            android:paddingTop="6dip"
            android:paddingBottom="2dip" />

        <TextView
            android:id="@+id/vehicleColor"
            android:layout_width="fill_parent"
           android:layout_height="wrap_content"
            android:paddingBottom="2dip">
        </TextView>

        <LinearLayout
            android:layout_width="fill_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal">

            <TextView
                android:id="@+id/fuel"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:gravity="left"
                android:textStyle="bold"
                android:text="Mobile: " >
            </TextView>
        </LinearLayout>
    </LinearLayout>

</LinearLayout>
```

Assim que a aplicação for aberta, ela irá consumir de imediato o JSON da BlackBerry e exibirá todo o conteúdo em uma lista. Portanto, na classe principal iremos implementar as funcionalidades abaixo.

Primeiramente, em nossa classe MainActivity.java, iremos acrescentar os imports abaixo para podermos utilizá-lo.

```
import android.app.ListActivity;
import android.app.ProgressDialog;
import android.content.Context;
import android.os.AsyncTask;
import android.os.Bundle;
import android.app.Activity;
import android.util.Log;
import android.view.Menu;
import android.widget.ListAdapter;
import android.widget.ListView;
import android.widget.SimpleAdapter;

import org.json.JSONArray;
import org.json.JSONException;
import org.json.JSONObject;

import java.util.ArrayList;
import java.util.HashMap;
```

Iremos criar uma classe dentro da classe MainActivity, que será responsável pela conexão com a URL do JSON. Esse código iremos disponibilizá-lo logo abaixo do método onCreate.

```
public class MainActivity extends ActionBarActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    private class ProgressTask extends AsyncTask<String, Void, Boolean> {
        private ProgressDialog dialog;

        public ProgressTask(ListActivity activity) {

            Log.i("1", "Called");
            context = activity;
            dialog = new ProgressDialog(context);
```

Figura 8

```
private class ProgressTask extends AsyncTask<String, Void,
Boolean> {
    private ProgressDialog dialog;

    public ProgressTask(ListActivity activity) {

        Log.i("1", "Called");
        context = activity;
        dialog = new ProgressDialog(context);
    }

    private Context context;

    protected void onPreExecute() {
        this.dialog.setMessage("Processo em andamento");
        this.dialog.show();
    }

    @Override
    protected void onPostExecute(final Boolean success) {
        if (dialog.isShowing()) {
            dialog.dismiss();
        }
        ListAdapter adapter = new SimpleAdapter(context,
jsonlist,
                R.layout.list_activity, new String[]
                { type, color, fuel }, new int[]
                { R.id.vehicleType, R.id.vehicleColor, R.id.
fuel });
        setListAdapter(adapter);
        lv = getListView();

    }

    protected Boolean doInBackground(final String... args) {

        JSONParser jParser = new JSONParser();
        JSONArray json = jParser.getJSONFromUrl(url);

        for (int i = 0; i < json.length(); i++) {
```

```
            try {
                JSONObject c = json.getJSONObject(i);
                String vtype = c.getString(type);

                String vcolor = c.getString(color);
                String vfuel = c.getString(fuel);

                HashMap<String, String> map = new
HashMap<String, String>();

                map.put(type, vtype);
                map.put(color, vcolor);
                map.put(fuel, vfuel);


                jsonlist.add(map);
            } catch (JSONException e)
            {
                e.printStackTrace();
            }
        }
        return null;
    }
}
```

Iremos alterar a classe MainActivity para estender a classe ListActivity, visto que iremos exibir isso em uma lista. Perceba, também, que abaixo criamos algumas variáveis no qual são a estrutura do JSON e o endereço do mesmo.

```
public class MainActivity extends ListActivity {

    private Context context;
    private static String url = "http://docs.blackberry.com/
sampledata.json";

    private static final String type = "vehicleType";
    private static final String color = "vehicleColor";
    private static final String fuel = "fuel";

    ArrayList<HashMap<String, String>> jsonlist = new
ArrayList<HashMap<String, String>>();
   ListView lv ;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        new ProgressTask(MainActivity.this).execute();
    }
```

Note que, no método onCreate(), foi chamado a classe ProgressTask(). Isso significa que, assim que abrirmos nossa app, iremos chamar em um nível de processamento paralelo para o conteúdo dessa página.

Deve estar ocorrendo um erro, pois precisamos criar a classe JSONParser; para isso, com o botão direito do mouse, crie uma nova classe com o nome acima.

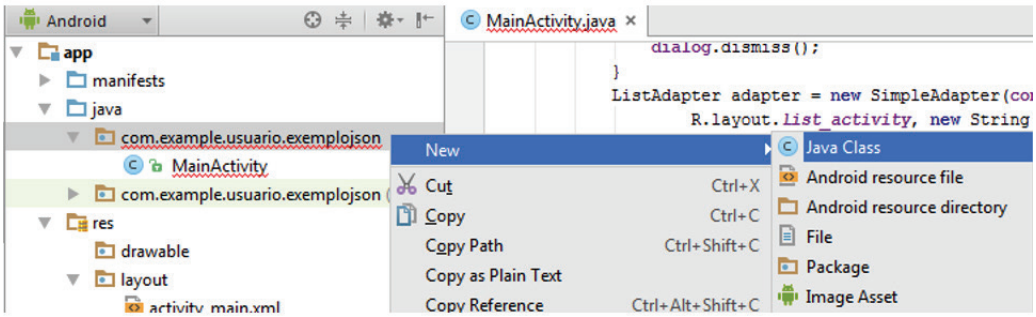


Figura 9

Substitua pelo código abaixo:

```
package com.example.usuario.exemplojson;

import android.util.Log;

import org.apache.http.HttpEntity;
import org.apache.http.HttpResponse;
import org.apache.http.StatusLine;
import org.apache.http.client.ClientProtocolException;
import org.apache.http.client.HttpClient;
import org.apache.http.client.methods.HttpGet;
import org.apache.http.impl.client.DefaultHttpClient;
import org.json.JSONArray;
import org.json.JSONException;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;

public class JSONParser {

    static InputStream is = null;
    static JSONArray jarray = null;
    static String json = "";

    public JSONParser() {
    }

    public JSONArray getJSONFromUrl(String url) {
```

```
        StringBuilder builder = new StringBuilder();
        HttpClient client = new DefaultHttpClient();
        HttpGet httpGet = new HttpGet(url);
        try {
            HttpResponse response = client.
execute(httpGet);
            StatusLine statusLine = response.getStatus-
Line();
            int statusCode = statusLine.getStatusCode();
            if (statusCode == 200) {
                HttpEntity entity = response.getEnti-
ty();
                InputStream content = entity.getCon-
tent();
                BufferedReader reader = new
BufferedReader(new InputStreamReader(content));
                String line;
                while ((line = reader.readLine()) !=
null) {
                    builder.append(line);
                }
            } else {
                Log.e("Erro....", "Falha no download do
arquivo");
            }
        } catch (ClientProtocolException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }

        try {
            jarray = new JSONArray( builder.toString());
        } catch (JSONException e) {
            Log.e("JSON Parser", "Erro no acesso aos da-
dos " + e.toString());
        }
        return jarray;
    }
}
```

Objetos Http

Construir um aplicativo que faça solicitações HTTP pode ser algo interessante. Muitas vezes, será suficiente utilizar o navegador (browser) e acessar sites, mas, em outras ocasiões, preferiremos um visual simplificado, personalizado ou mais ajustado à tela dos aparelhos. Existem muitos exemplos de aplicativos Android que se comunicam via Web, dentre eles as versões de Yahoo Mail, Skype, Fring, Facebook etc., que apresentam um visual mais cômodo e adequado para acessar estas aplicações.

Uma das características que definem os sistemas móveis modernos é que eles podem nos manter conectado e em rede, sem nos prender a um local e nesta aula, iremos explorar as práticas que você precisa para conectar seus aplicativos na internet, portanto vamos começar esta aula, discutindo redes em geral.

Essa discussão se concentrará em conectar suas aplicações à internet usando Hypertext Transfer Protocol ou HTTP, especificamente usando solicitações HTTP GET.

Os dispositivos de hoje combinam poderosos processadores com conexões de rede Wi-Fi mais rápidas e redes móveis. Aplicações portáteis, portanto, muitas vezes querem fazer uso desses recursos de rede para acessar e fornecer dados e serviços. O Android inclui uma variedade de classes de suporte incluindo Socket e URL, no pacote Java.net. As classes HttpRequest e HttpResponse nos pacotes org.apache, nos auxiliam para fazer isso, sendo as classes URI, AndroidHttpClient e AudioStream dos pacotes android.net.

No código acima, o código do status 200 representa que a página foi encontrada.

Execute a aplicação e perceba que, ao carregá-la, será exibida a mensagem abaixo:



Figura 10

E, logo em seguida, será exibida a lista com o JSON acima comentado.

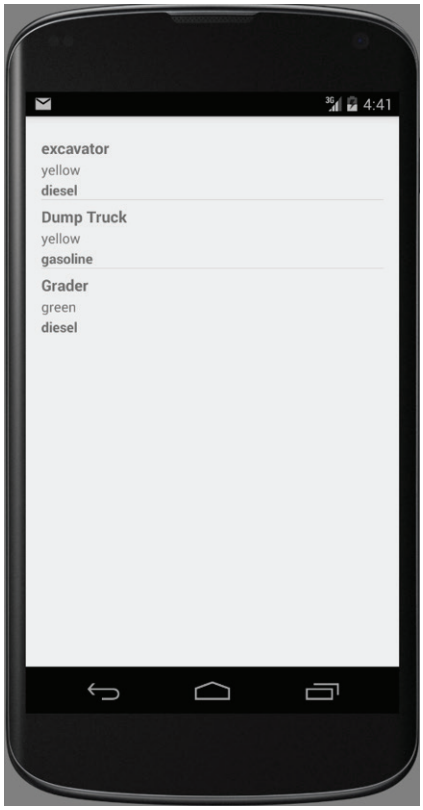


Figura 11

Conclusão

Percebemos que o JSON é um a estrutura bem simples e leve e a lógica para se consumir outro JSON é a mesma, basta saber quais parâmetros você vai querer consumir e isso pode ser verificado na própria página da web onde o JSON está disponível.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Para aprofundar seus estudos sobre gráficos consulte o *site*:

Vídeos

Parser JSON no Android, Entendendo e Utilizando
https://youtu.be/XN_RpT7XqFM

Android Tutorial. Conectar a web service mediante Android con JSON.
https://youtu.be/GipKwMGO6Nc

Android REST web service tutorial with JSON
https://youtu.be/lU7t5BoxxOA

Referências

ANDROID. **Android Developers**. Disponível em http://developer.android.com

LECHETA, R. R. **Google Android, Aprenda a criar aplicações para dispositivos móveis com o Android SDK**. São Paulo: Novatec, 2010.

MEDNIEKS, Z. et. al. **Desenvolvimento de Aplicações Android**. São Paulo: Novatec, 2009.

LEE, W. M. **Introdução ao Desenvolvimento de Aplicativos para o Android**. Rio de Janeiro: Ciência Moderna, 2011.

Guia do desenvolvedor:
http://docs.blackberry.com/en/developers/deliverables/25220/Code_sample_Parse_JSON_data_structure_1319797_11.jsp

Cruzeiro do Sul Virtual
Educação a Distância

www.cruzeirodosulvirtual.com.br
Campus Liberdade
Rua Galvão Bueno, 868
CEP 01506-000
São Paulo - SP - Brasil
Tel: (55 11) 3385-3000


Cruzeiro do Sul
Educacional